Diagramas de Objetos

2006
©MJRL

Engenharia de Software

Nota

- O material apresentado nos slides seguintes foram, na sua maioria, extraídos de material disponibilizado por Eduardo Bezerra no site *sites.uol.com.br/eduardo.bezerra/papoo,* como suporte ao seu livro “Princípios de Análise e Projeto de Sistemas com UML” publicado pela editora Campus.

Introdução

- O modelo de casos de uso fornece uma perspectiva do sistema a partir de um ponto de vista ***externo***.
- De posse da visão de casos de uso, os desenvolvedores precisam prosseguir no desenvolvimento do sistema.

2006
©MJRL

Aspectos dinâmico e estático

- A funcionalidade externa de um sistema orientado a objetos é fornecida através de colaborações entre objetos.
  - Externamente, os atores visualizam resultados de cálculos, relatórios produzidos, confirmações de requisições realizadas, etc.
  - Internamente, os objetos colaboram uns com os outros para produzir os resultados.
- Essa colaboração pode ser vista sob o ***aspecto dinâmico*** e sob o ***aspecto estrutural estático***.

2006
©MJRL

Modelo de classes

- O diagrama da UML utilizado para representar o aspecto estático é o ***diagrama de classes***.
- O ***modelo de classes*** é composto desse diagrama e da descrição textual associada.
- Objetivos principais deste módulo:
  - Descrever um método para identificação das classes de objetos de um sistema partir do modelo de casos de uso.
  - Apresentar alguns dos elementos do diagrama de classes (outros elementos são descritos em capítulos posteriores).
  - Descrever a construção do modelo de domínio.
  - Descrever a inserção do modelo de classes no processo de desenvolvimento.

2006
©MJRL

Modelo de classes

- O modelo de classes evolui durante o desenvolvimento do sistema.
  - À medida que o sistema é desenvolvido, o modelo de classes é incrementado com novos detalhes.
- Três *níveis sucessivos de abstração:*
  - *Domínio*
  - *Especificação*
  - *Implementação*.

2006
©MJRL

Modelo de classes

- **O *modelo de classes de domínio*** representa as classes no domínio do negócio em questão. Não leva em consideração restrições inerentes à tecnologia a ser utilizada na solução de um problema.
- **O *modelo de classes de especificação*** é obtido através da adição de detalhes ao modelo anterior conforme a solução de software escolhida.
- **O *modelo de classes de implementação*** corresponde à implementação das classes em alguma linguagem de programação.

2006
©MJRL

Modelo de classes de domínio

- Representa termos do domínio do negócio.
- Objetivo:
  - descrever o *problema* representado pelo sistema a ser desenvolvido, sem considerar características da *solução* a ser utilizada.

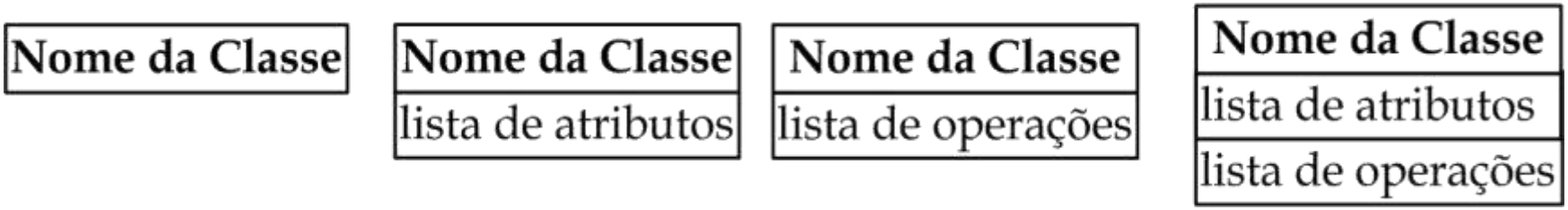
Classes

- Uma classe representa um grupo de objetos semelhantes.
- Uma classe descreve esses objetos através de ***atributos*** e ***operações***.
- Os atributos correspondem às informações que um objeto armazena.
- As operações correspondem às ações que um objeto sabe realizar.

2006
©MJRL

Notação para uma classe

- Representada através de uma “caixa” com no máximo três compartimentos exibidos.
- Notação utilizada depende do nível de abstração desejado.



Engenharia de Software

Exemplo (classe ContaBancária)



Associações

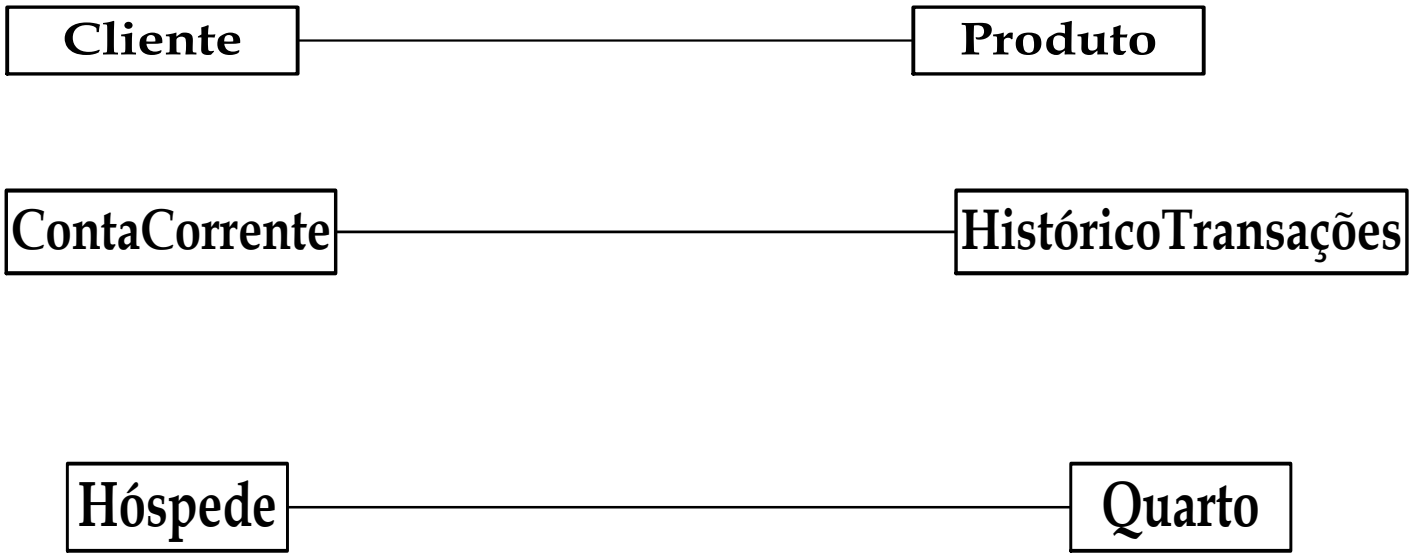
- Para representar o fato de que objetos podem se relacionar uns com os outros, utiliza-se a *associação*.
- Uma associação representa relacionamentos (ligações)que são formados entre objetos durante a execução do sistema.
  - embora as associações sejam representadas entre classes do diagrama, tais associações representam ligações possíveis entre *objetos* das classes envolvidas.

2006
©MJRL

Engenharia de Software

Notação para uma associação

- Representada através de um segmento de reta ligando as classes cujos objetos se relacionam.
- Exemplos:



Multiplicidades

- Representam a informação dos limites inferior e superior da quantidade de objetos aos quais um outro objeto pode estar associado.
- Cada associação em um diagrama de classes possui duas multiplicidades, uma em cada extremo da linha de associação.

2006
©MJRL

Engenharia de Software

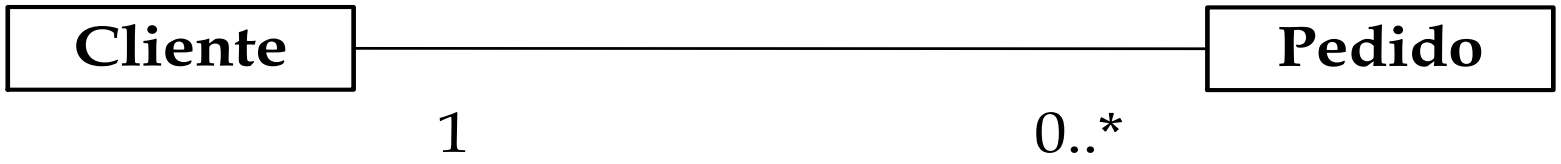
Multiplicidades

Nome	Simbologia
Apenas Um	1..1 (ou 1)
Zero ou Muitos	0..* (ou *)
Um ou Muitos	1..*
Zero ou Um	0..1
Intervalo Específico	$l_i..l_s$

2006
©MJRL

Exemplo (multiplicidade)

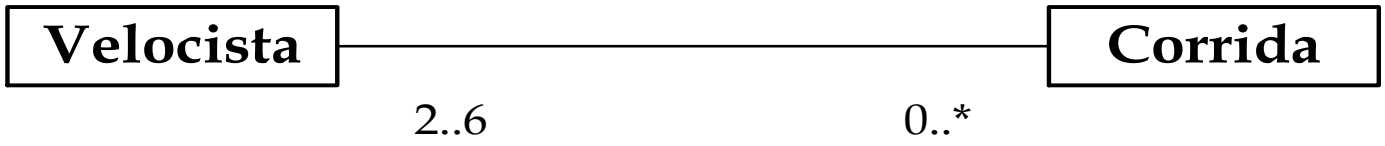
- Pode haver um cliente que esteja associado a vários pedidos.
- Pode haver um cliente que não esteja associado a pedido algum.
- Um pedido está associado a um, e somente um, cliente.



2006
©MJRL

Exemplo (multiplicidade)

- Uma corrida está associada a, no mínimo, dois velocistas
- Uma corrida está associada a, no máximo, seis velocistas.
- Um velocista *pode* estar associado a várias corridas.



2006
©MJRL

Conectividade

- A **conectividade** corresponde ao tipo de associação entre duas classes: "*muitos para muitos*", "*um para muitos*" e "*um para um*".
- A conectividade da associação entre duas classes depende dos símbolos de multiplicidade que são utilizados na associação.

©MJRL

Engenharia de Software

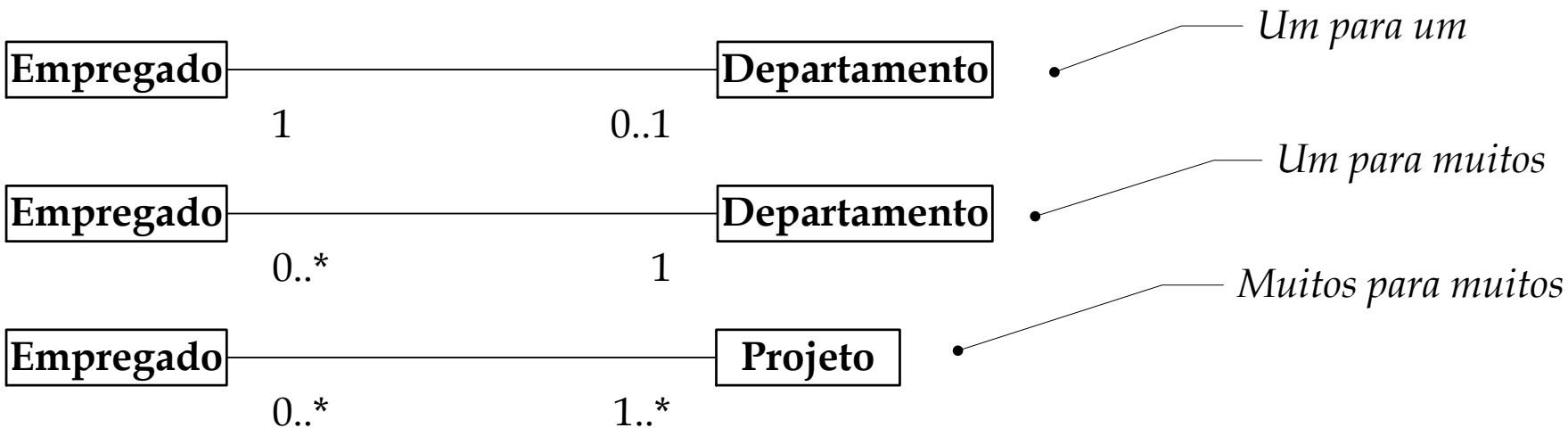
Conectividade X Multiplicidade

Conectividade	Em um extremo	No outro extremo
Um para um	0..1 1	0..1 1
Um para muitos	0..1 1	* 1..* 0..*
Muitos para muitos	* 1..* 0..*	* 1..* 0..*

2006
©MJRL

Engenharia de Software

Exemplo (conectividade)



Participação

- Uma característica de uma associação que indica a necessidade (ou não) da existência desta associação entre objetos.
- A participação pode ser *obrigatória* ou *opcional*.
  - Se o valor mínimo da multiplicidade de uma associação é igual a 1 (um), significa que a participação é obrigatória
  - Caso contrário, a participação é opcional.

2006
©MJRL

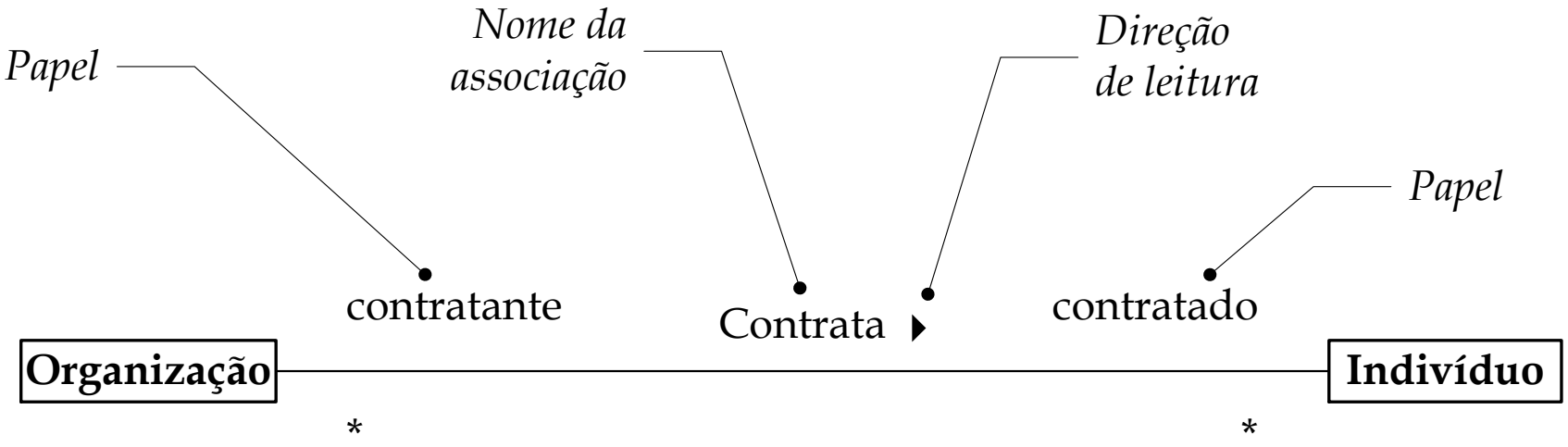
Nome de associação, direção de leitura e papéis

- Para melhor esclarecer o significado de uma associação no diagrama de classes, a UML define três recursos de notação:
  - ***Nome da associação***: fornece algum significado semântico a mesma.
  - ***Direção de leitura***: indica como a associação deve ser lida
  - ***Papel***: para representar um papel específico em uma associação.

2006
©MJRL

Engenharia de Software

Exemplo (Nome de associação, direção de leitura e papéis)



Agregação

- É um caso especial da associação
  - conseqüentemente, multiplicidades, participações, papéis, etc. podem ser usados igualmente
- Utilizada para representar conexões que guardam uma relação *todo-parte* entre si.
- Em uma agregação, um objeto está contido no outro, ao contrário de uma associação.
- Onde se puder utilizar uma agregação, uma associação também poderá ser utilizada.

2006
©MJRL

Agregação

- Características particulares:
  - Agregações são assimétricas: se um objeto A é parte de um objeto B, B não pode ser parte de A.
  - Agregações propagam comportamento, no sentido de que um comportamento que se aplica a um todo automaticamente se aplica as suas partes.

2006
©MJRL

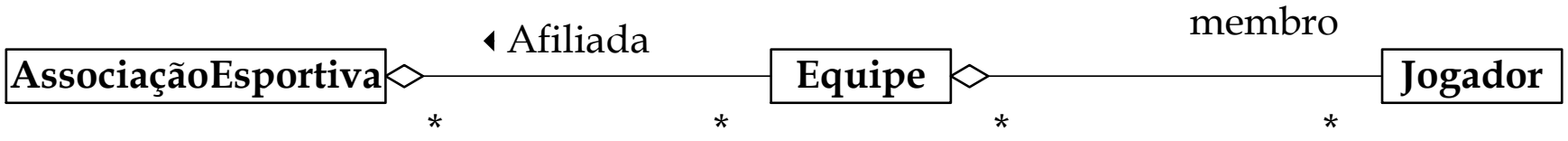
Agregação

- Sejam duas classes associadas, X e Y. Se uma das perguntas a seguir for respondida com um sim, provavelmente há uma agregação onde X é todo e Y é parte.
  - *X tem um ou mais Y?*
  - *Y é parte de X?*

2006
©MJRL

Notação para uma agregação

- *Representada como uma linha conectando as classes relacionadas, com um diamante (losango) branco perto da classe que representa o todo.*
- Exemplo:



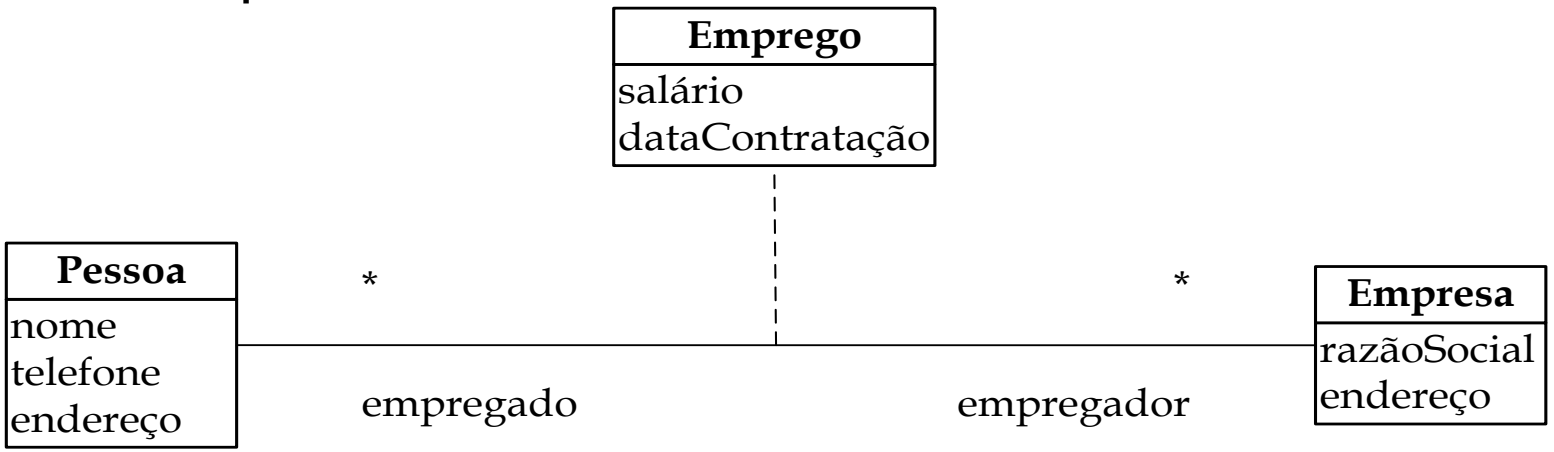
Classe associativa

- É uma classe que está ligada a uma associação, ao invés de estar ligada a outras classes.
- É normalmente necessária quando duas ou mais classes estão associadas, e é necessário manter informações sobre esta associação.
- Uma classe associativa pode estar ligada a associações de qualquer tipo de conectividade.

2006
©MJRL

Notação para uma classe associativa

- *Representada pela notação utilizada para uma classe. A diferença é que esta classe é ligada a uma associação.*
- Exemplo:



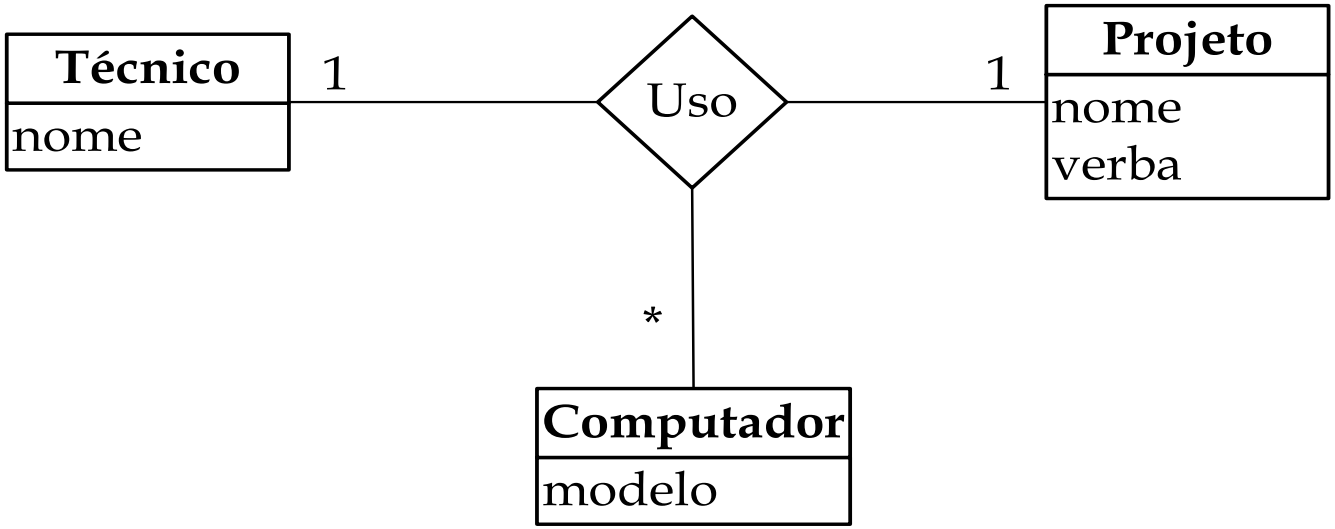
Associações n-árias

- São utilizadas para representar a associação existente entre objetos de n classes.
- Uma ***associação ternária*** são uma caso mais comum (menos raro) de associação n-ária (n = 3).
- Na notação da UML, as linhas de associação se interceptam em um losango.

2006
©MJRL

Engenharia de Software

Exemplo (associação ternária)

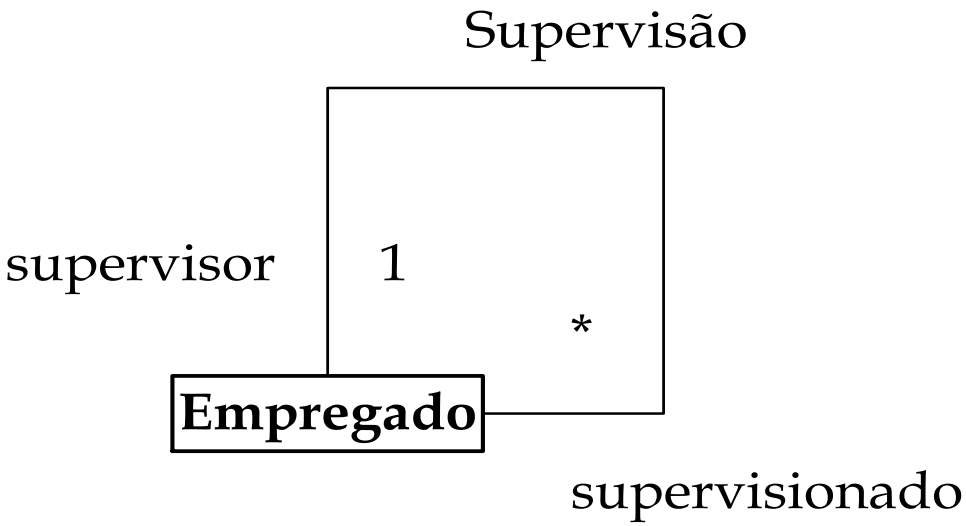


Associações reflexivas

- Associa objetos da mesma classe.
  - Cada objeto tem um papel distinto na associação.
- A utilização de papéis é bastante importante para evitar ambigüidades na leitura da associação.
- Uma associação reflexiva *não* indica que um objeto se associa com ele próprio.
  - Ao contrário, indica que objetos de uma mesma classe se associam

2006
©MJRL

Exemplo (associação reflexiva)



Identificando as classes iniciais

2006
©MJRL

Identificando classes

- Um sistema de software orientado a objetos é composto de objetos em colaboração para realizar as tarefas deste sistema.
- Por outro lado, todo objeto pertence a uma classe.
- Portanto, quando se fala na identificação das classes, o objetivo na verdade é saber quais objetos irão compor o sistema.

2006
©MJRL

Identificando classes

- De uma forma geral, a identificação de classes se divide em duas atividades.
  - Primeiramente, ***classes candidatas*** são identificadas.
  - Depois disso, são aplicados alguns princípios para eliminar classes candidatas desnecessárias.
- Identificar *possíveis* classes para um sistema não é complicado; o difícil é eliminar deste conjunto o que não é necessário.

2006
©MJRL

Identificação dirigida a responsabilidades

- Método de identificação onde a ênfase está na identificação de classes a partir de seus *comportamentos* externos relevantes para o sistema.
  - *como* a classe faz para cumprir com suas responsabilidades deve ser abstraído.
- O esforço recai sobre a identificação das *responsabilidades* que cada classe deve ter.
- "O método *dirigido a responsabilidades enfatiza o encapsulamento da estrutura e do comportamento dos objetos.*".

2006
©MJRL

Responsabilidades e colaboradores

- Em sistemas OO, objetos encapsulam tanto dados quanto comportamento.
- O comportamento de um objeto é definido de tal forma que ele possa cumprir com suas ***responsabilidades***.
- Uma responsabilidade é uma obrigação que um objeto tem para com o sistema no qual ele está inserido.
  - Através delas, um objeto colabora (ajuda) com outros para que os objetivos do sistema sejam alcançados.

2006
©MJRL

Responsabilidades e colaboradores

- Na prática, uma responsabilidade é alguma coisa que um objeto *conhece* ou *faz. (sozinho ou não)*.
  - Um objeto Cliente *conhece* seu nome, seu endereço, seu telefone, etc.
  - Um objeto Pedido *conhece* sua data de realização e sabe *fazer* o cálculo do seu total.
- Se um objeto tem uma responsabilidade com a qual não pode cumprir sozinho, ele deve requisitar ***colaborações*** de outros objetos.

2006
©MJRL

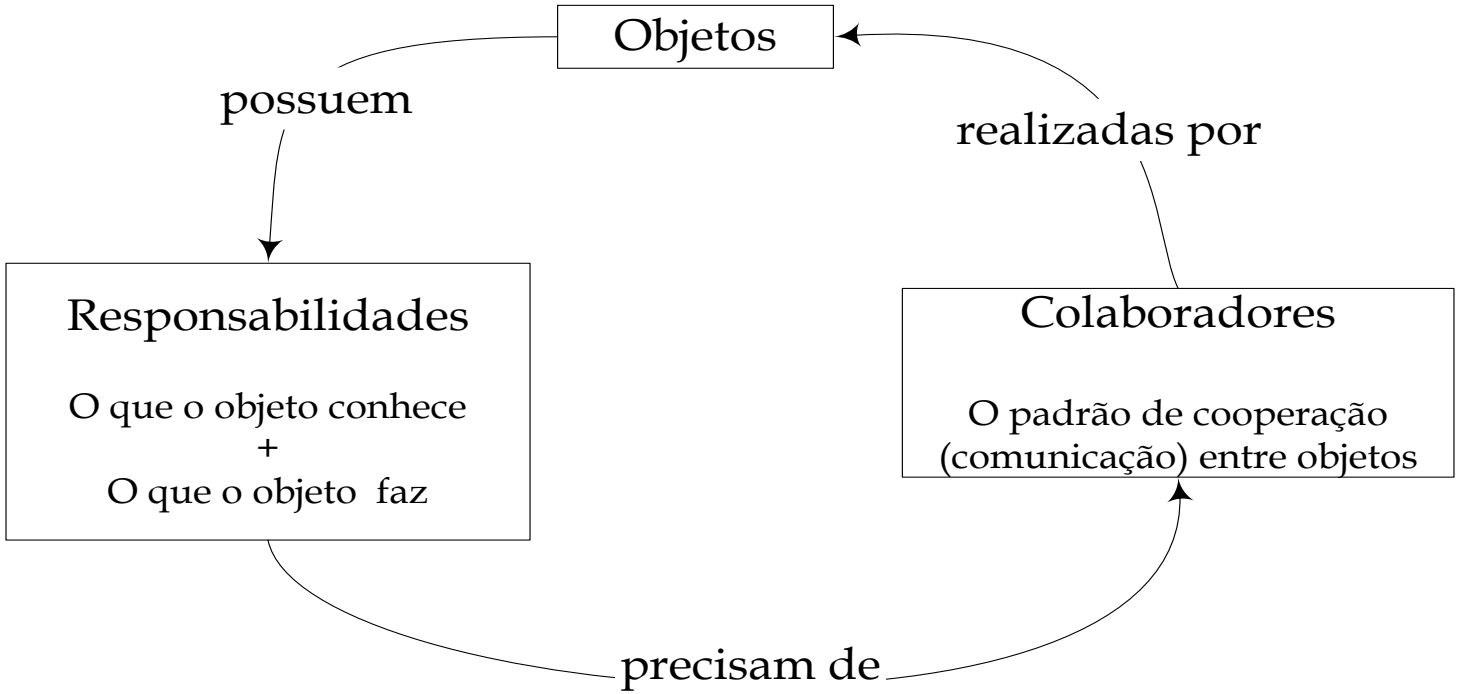
Responsabilidades e colaboradores

- Um exemplo: quando a impressão de uma fatura é requisitada em um sistema de vendas, vários objetos precisam colaborar:
  - um objeto Pedido pode ter a responsabilidade de fornecer o seu valor total
  - um objeto Cliente fornece seu nome
  - cada ItemPedido informa a quantidade correspondente e o valor de seu subtotal
  - os objetos Produto também colaboraram fornecendo seu nome e preço unitário.

2006
©MJRL

Engenharia de Software

Responsabilidades e colaboradores



Categorias de responsabilidades

- Costuma-se categorizar os objetos de um sistema de acordo com o tipo de responsabilidade a ele atribuída.
  - objetos de entidade
  - objetos de controle
  - objetos de fronteira
- Esta categorização foi proposta por Ivar Jacobson (Jacobson et al, 1992) em uma técnica denominada Análise de Robustez.

2006
©MJRL

Engenharia de Software

Objetos de Entidade



- Um objeto de entidade é um repositório para alguma ***informação*** manipulada pelo sistema.
- Esses objetos representam conceitos do domínio do negócio.
- Normalmente esses objetos armazenam informações persistentes.
- Há várias instâncias de uma mesma classe de entidade coexistindo no sistema.

2006
©MJRL

Objetos de Entidade

- Atores não têm acesso direto a estes objetos.
  - Objetos de entidade se comunicam com o exterior do sistema por intermédio de outros objetos.
- Objetos de entidade normalmente participam de vários casos de uso e têm um ciclo de vida longo.
  - Um objeto *Pedido* pode participar dos casos de uso *Realizar Pedido* e *Atualizar Estoque*. Este objeto pode existir por diversos anos ou mesmo tanto quanto o próprio sistema.

2006
©MJRL

Objetos de Entidade

- Responsabilidades de fazer típicas de objetos de entidade:
  - Informar valores de seus atributos a objetos de controle.
  - Realizar cálculos simples, normalmente com a colaboração de objetos de entidade associados através de agregações.
  - Criar e destruir objetos parte (considerando que o objeto de entidade seja um objeto todo de uma agregação).

2006
©MJRL

Objetos de Fronteira



- Esses objetos traduzem os eventos gerados por um ator em eventos relevantes ao sistema.
- Também são responsáveis por apresentar os resultados de uma interação dos objetos em algo inteligível pelo ator.
- Um objeto de fronteira existe para que o sistema se comunique com o mundo exterior.
- Por conseqüência, estes objetos são altamente dependentes do ambiente.

2006
©MJRL

Objetos de Fronteira

- Classes de fronteira realizam a comunicação do sistema com atores, sejam eles outros sistemas, equipamentos ou seres humanos.
- Há três tipos principais de classes de fronteira:
  - as que se comunicam com o usuário (atores humanos),
  - as que se comunicam com outros sistemas
  - as que se comunicam com dispositivos atrelados ao sistema.

2006
©MJRL

Objetos de Fronteira

- Tipicamente têm as seguintes responsabilidades *de fazer*:
  - Notificar aos objetos de controle de eventos gerados externamente ao sistema.
  - Notificar aos atores do resultado de interações entre os objetos internos.

2006
©MJRL

Objetos de Fronteira

- Responsabilidades *de conhecer* de classes de fronteira para interação humana representam informação manipulada através da interface com o usuário.
  - A construção de protótipos pode ajudar a identificar essas responsabilidades.
- Responsabilidades de conhecer para classes de fronteira que realizam comunicação com outros sistemas representam propriedades de uma interface de comunicação.

2006
©MJRL

Objetos de Controle



- São a “ponte de comunicação” entre objetos de fronteira e objetos de entidade.
- Responsáveis por controlar a lógica de execução correspondente a um caso de uso.
- Decidem o que o sistema deve fazer quando um evento externo relevante ocorre.
  - Eles realizam o controle do processamento
  - Agem como ***gerentes*** (coordenadores, controladores) dos outros objetos para a realização de um ou mais caso de uso.

2006
©MJRL

Objetos de Controle

- São bastante acoplados à lógica da aplicação.
- Traduzem eventos externos em operações que devem ser realizadas pelos demais objetos.
- Ao contrário dos objetos de entidade e de fronteira, objetos de controle são tipicamente ativos
  - consultam informações e requisitam serviços de outros objetos.

2006
©MJRL

Objetos de Controle

- Responsabilidades de fazer típicas:
  - Realizar monitorações, a fim de responder a eventos externos ao sistema (gerados por objetos de fronteira).
  - Coordenar a realização de um caso de uso através do envio de mensagens a objetos de fronteira e objetos de entidade.
  - Assegurar que as regras do negócio (Seção 4.5.1) estão sendo seguidas corretamente.
  - Coordenar a criação de associações entre objetos de entidade.

2006
©MJRL

Objetos de Controle

- Responsabilidades de conhecer estão associadas manter valores acumulados, temporários ou derivados durante a realização de um caso de uso.
- Podem também ter o objetivo de manter o estado da realização do caso de uso.
- Têm vida curta: normalmente existem somente durante a realização de um caso de uso.

2006
©MJRL

Divisão de responsabilidades

- A categorização de responsabilidades implica em que cada objeto é especialista em realizar um de três tipos de tarefa:
  - se comunicar com atores (fronteira)
  - manter as informações do sistema (entidade)
  - coordenar a realização de um caso de uso (controle).
- A importância dessa categorização está relacionada à capacidade de adaptação do sistema a eventuais mudanças

2006
©MJRL

Divisão de responsabilidades

- Se cada objeto tem funções específicas dentro do sistema, eventuais mudanças no sistema podem ser:
  1. menos complexas
  2. mais localizadas.
- Uma eventual modificação em uma parte do sistema tem menos possibilidades de resultar em mudanças em outras partes.

2006
©MJRL

Engenharia de Software

Divisão de responsabilidades

Tipo de mudança	Onde mudar
Mudanças em relação à interface gráfica, ou em relação à comunicação com outros sistemas.	Fronteira
Mudanças nas informações manipuladas pelo do sistema	Entidade
Mudanças em funcionalidades complexas (lógica do negócio)	Controle

2006
©MJRL

Divisão de responsabilidades

- Exemplo: vantagem de separação de responsabilidades em um sistema para uma loja de aluguel de carros.
  - Se o sistema tiver que ser atualizado para que seus usuários possam utilizá-lo pela Internet, a lógica da aplicação não precisaria de modificações.
  - Considerando a lógica para calcular o valor total das locações feitas por um cliente: se esta lógica estiver encapsulada em uma classe de controle, somente esta classe precisaria de modificação.

2006
©MJRL

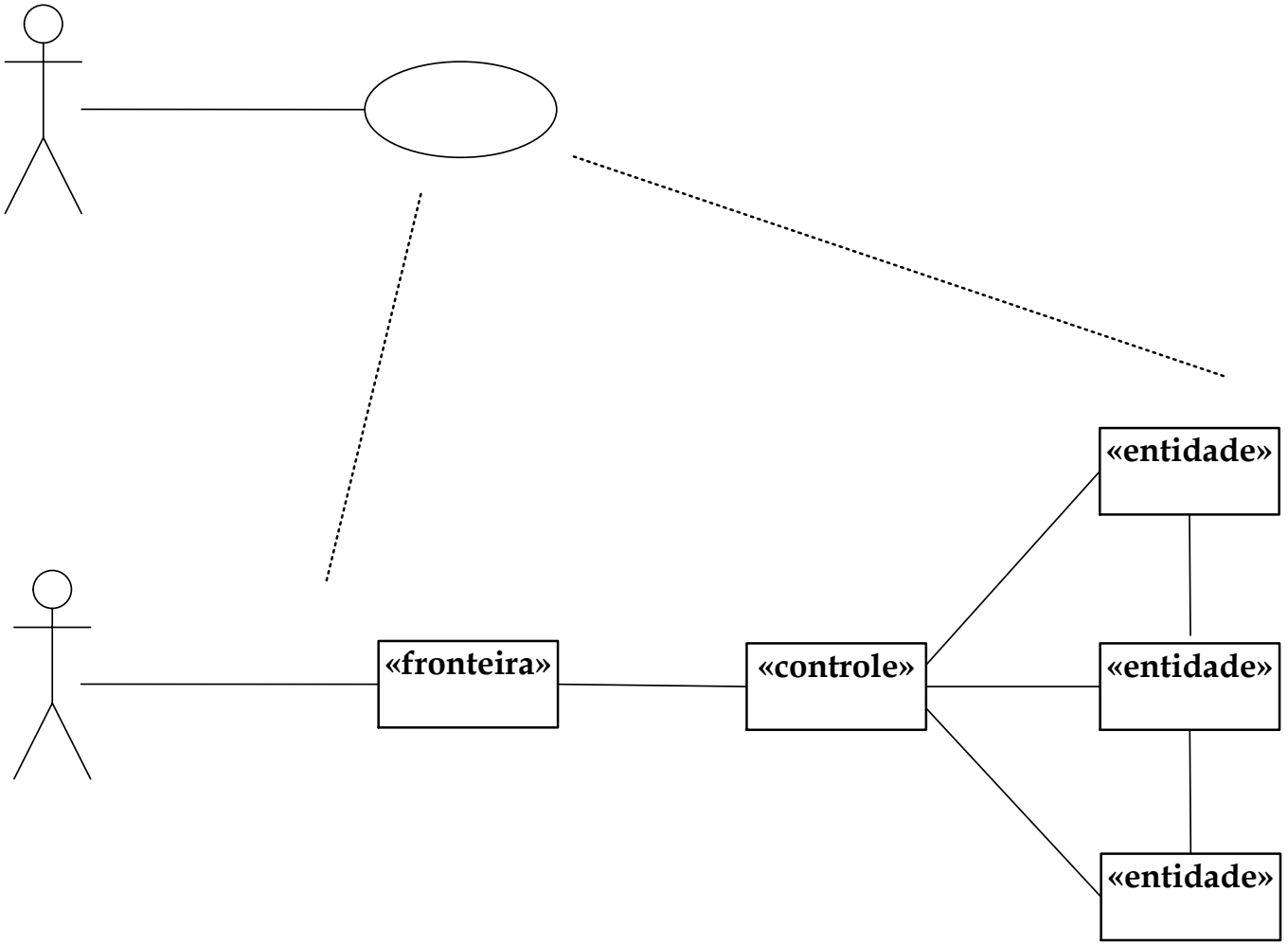
Divisão de responsabilidades

- A construção de um sistema de software que faça separação das responsabilidades de apresentação (fronteira), de lógica da aplicação (controle) e de manutenção dos dados (entidade):
  - facilita também o reuso dos objetos no desenvolvimento de sistemas de software semelhantes.
  - ajuda no desacoplamento entre elementos do sistema

2006
©MJRL

Engenharia de Software

Divisão de responsabilidades



Partindo para a identificação

- ***Análise os casos de uso***: cada caso de uso é analisado para identificar classes candidatas.
- Premissa: a partir da descrição textual dos casos de uso, podem-se derivar as classes do sistema.
  - a existência de uma classe em um sistema só pode se justificar se ela participa de alguma forma para o comportamento externamente visível do sistema.

2006
©MJRL

Partindo para a identificação

- Os substantivos que aparecem no texto do caso de uso são destacados.
  - São também consideradas locuções equivalentes a substantivos.
- Sinônimos são removidos.
- Vantagem: abordagem é bastante simples.
- Desvantagem: depende de como os casos de uso foram escritos.
  - em linguagem natural, as formas de expressar uma mesma idéia são bastante numerosas.

2006
©MJRL

Partindo para a identificação

- Para contornar os problemas na identificação de classes através da análise de casos de uso, uma solução é aplicar uma estratégia em dois passos.
  1. Faz-se a análise dos casos de uso para identificar as classes candidatas.
  2. Depois disso, aplica-se uma outra técnica para validar o que foi descoberto e para identificar novas classes: a ***modelagem CRC***.

2006
©MJRL

Modelagem CRC

- Se baseia fortemente no paradigma da orientação a objetos, onde objetos cooperam uns com os outros para que uma tarefa do sistema seja realizada.
- Efetiva quando profissionais que não têm tanta experiência com o paradigma da orientação a objetos estão envolvidos na identificação de classes.
  - realizada em conjunto por especialistas de domínio e desenvolvedores

2006
©MJRL

Modelagem CRC

- A modelagem CRC não faz parte da UML.
- A princípio, essa técnica foi proposta como uma forma de ensinar o paradigma da orientação a objetos a iniciantes.
- Contudo, a sua simplicidade de notação a tornou particularmente interessante para ser utilizada na identificação de classes de domínio.

2006
©MJRL

Modelagem CRC

- Especialistas do negócio e desenvolvedores trabalham em conjunto para identificar classes, juntamente com suas responsabilidades e colaboradores.
- Estes profissionais se reúnem em uma sala, onde tem início uma ***sessão CRC***.
- Uma sessão CRC envolve por volta de meia dúzia de pessoas: especialistas de domínio, projetistas, analistas e um moderador.
- A cada pessoa é entregue um ***cartão CRC***.

2006
©MJRL

Engenharia de Software

Cartão CRC

Nome da classe (especialidade)	
Responsabilidades	**Colaboradores**
1ª responsabilidade 2ª responsabilidade ... n-ésima responsabilidade	1° colaborador 2° colaborador .. n-ésimo colaborador

2006
©MJRL

Engenharia de Software

Exemplo (Cartão CRC)

ContaBancária (entidade)	
Responsabilidades	**Colaboradores**
1.Conhecer o seu cliente. 2.Conhecer o seu número. 3.Conhecer o seu saldo. 4.Manter um histórico de transações. 5.Aceitar saques e depósitos.	Cliente HistóricoTransações

2006
©MJRL

Modelagem CRC

- Na distribuição dos cartões pelos participantes, deve-se considerar as categorias de responsabilidades.
- Para cada cenário de caso de uso típico, pode-se começar com:
  - um objeto de fronteira para cada ator participante do caso de uso;
  - um objeto de controle para todo o caso de uso;
  - normalmente há vários objetos de entidade.

2006
©MJRL

Modelagem CRC

- Configuração inicial:
  - O moderador da sessão pode desempenhar o papel do objeto controlador
  - Outro participante desempenha o papel do objeto de fronteira.
  - Um outro participante pode simular o ator (ou atores, se houver mais de um).
  - Os demais representam objetos de entidade.

2006
©MJRL

Modelagem CRC

- Uma vez distribuídos os cartões pelos participantes, um conjunto de cenários de cada caso de uso é selecionado.
- Para cada cenário, uma sessão CRC é realizada.
  - Se o caso de uso não for tão complexo, ele pode ser analisado em uma única sessão.
- Normalmente já existem algumas classes candidatas para um certo cenário.

2006
©MJRL

Modelagem CRC

- A sessão CRC começa com a simulação do ator primário disparando a realização do caso de uso.
- Os demais participantes encenam a colaboração entre objetos para que o objetivo do ator seja alcançado.
- Através dessa encenação, as classes, responsabilidades e colaborações são identificadas.

2006
©MJRL

Modelagem CRC (procedimento)

1. Selecionar um conjunto de cenários de casos de uso.
2. Para um dos cenários:
   a) Examinar a sua seqüência de passos para identificar as responsabilidades do sistema em relação a cada um desses passos.
   b) Identificar classes relevantes que devem cumprir com as responsabilidades.
3. Repetir o passo 2 para o próximo cenário e modificar a alocação de responsabilidades e a definição de classes.

2006
©MJRL

Dicas para atribuição de responsabilidades

- Associar responsabilidades com base na especialidade da classe.
- Distribuir a inteligência do sistema
- Agrupar as responsabilidades conceitualmente relacionadas
- Evitar responsabilidades redundantes

2006
©MJRL

Engenharia de Software

Construção do modelo de classes de domínio

Propriedades de uma classe

- Uma responsabilidade de conhecer é mapeada para um ou mais atributos.
- Operações de uma classe são um modo mais detalhado de explicitar as responsabilidades de fazer.
  - Uma operação pode ser vista como uma contribuição da classe para uma tarefa mais complexa representada por um caso de uso.
  - Uma definição mais completa das operações de uma classe só pode ser feita após a construção dos **diagramas de interação**.

2006
©MJRL

Definição de associações e agregações

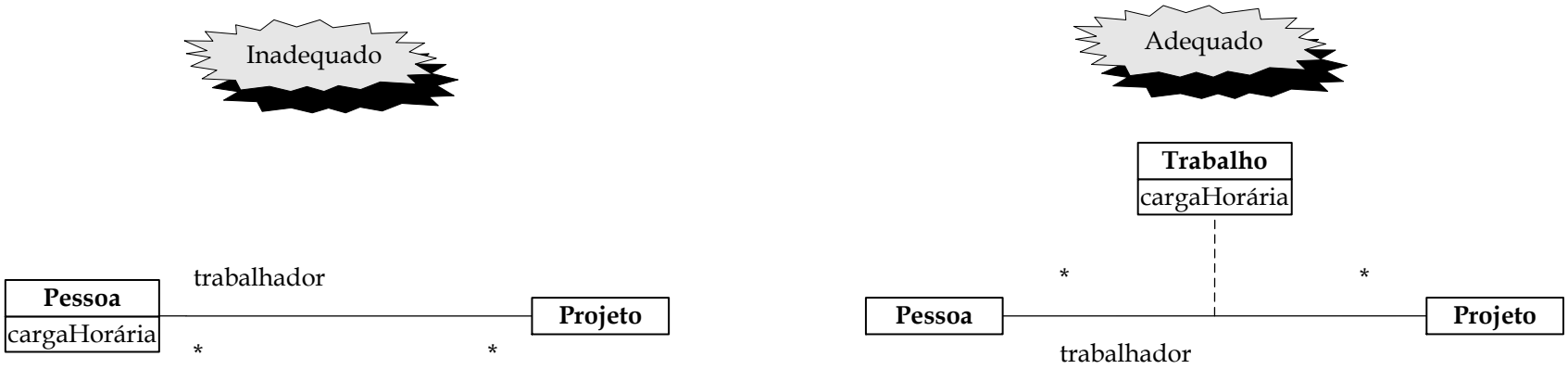
- O fato de uma classe possuir colaboradores indica que devem existir relacionamentos entre estes últimos e a classe.
  - Isto porque um objeto precisa conhecer o outro para poder lhe fazer requisições.
  - Portanto, para criar associações, verifique os colaboradores de uma classe.
- O raciocínio para definir associações reflexivas, ternárias e agregações é o mesmo.

2006
©MJRL

Engenharia de Software

Definição de classes associativas

- Surgem a partir de responsabilidades de conhecer que o modelador não conseguiu atribuir a alguma classe.
  - (mais raramente, de responsabilidades *de fazer)*



Documentando o modelo de classes



- As responsabilidades e colaboradores mapeados para elementos do modelo de classes devem ser organizados em um diagrama de classes e documentados, resultando no ***modelo de classes de domínio***.
- Podem ser associados estereótipos predefinidos às classes identificadas:
  - <<fronteira>>
  - <<entidade>>
  - <<controle>>

Documentando o modelo de classes

- A construção de um único diagrama de classes para todo o sistema pode resultar em um diagrama bastante complexo. Um alternativa é criar uma ***visão de classes participantes*** (VCP) para cada caso de uso.
- Em uma VCP, são exibidos os objetos que participam de um caso de uso.
- As VCPs podem ser reunidas para formar um único diagrama de classes para o sistema como um todo.

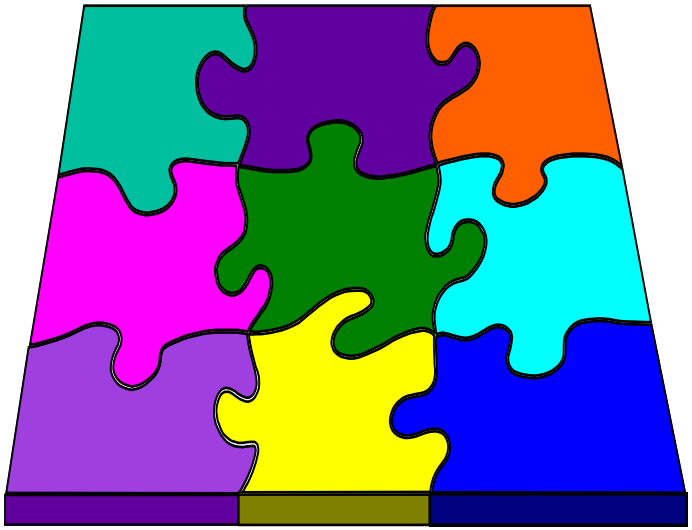
2006
©MJRL

Documentando o modelo de classes

- O modelador pode optar por “esconder” as classes de fronteira ou até mesmo as classes de controle.
  - Uma ferramenta CASE que dê suporte a essa operação seria de grande ajuda para a equipe de desenvolvimento.
- Além do diagrama, as classes devem ser documentadas textualmente.

2006
©MJRL

Engenharia de Software



Modelo de classes no processo de desenvolvimento

2006
©MJRL

Engenharia de Software

Modelo de classes no processo de desenvolvimento

- Em um desenvolvimento dirigido a casos de uso, após a descrição dos casos de uso, é possível iniciar a identificação de classes.
- As classes identificadas são refinadas para retirar inconsistências e redundâncias.
- As classes são documentadas e o diagrama de classes inicial é construído, resultando no modelo de classes de domínio.

2006
©MJRL

Modelo de classes no processo de desenvolvimento

- Inconsistências nos modelos devem ser verificadas e corrigidas.
- As construções do modelo de casos de uso e do modelo de classes são retroativas uma sobre a outra.
  - Na realização de uma sessão CRC, novos casos de uso podem ser identificados
  - Pode-se identificar a necessidade de modificação de casos de uso preexistentes.

2006
©MJRL

Engenharia de Software

Modelo de classes no processo de desenvolvimento

- Detalhes são adicionados aos modelos, à medida que o problema é entendido.

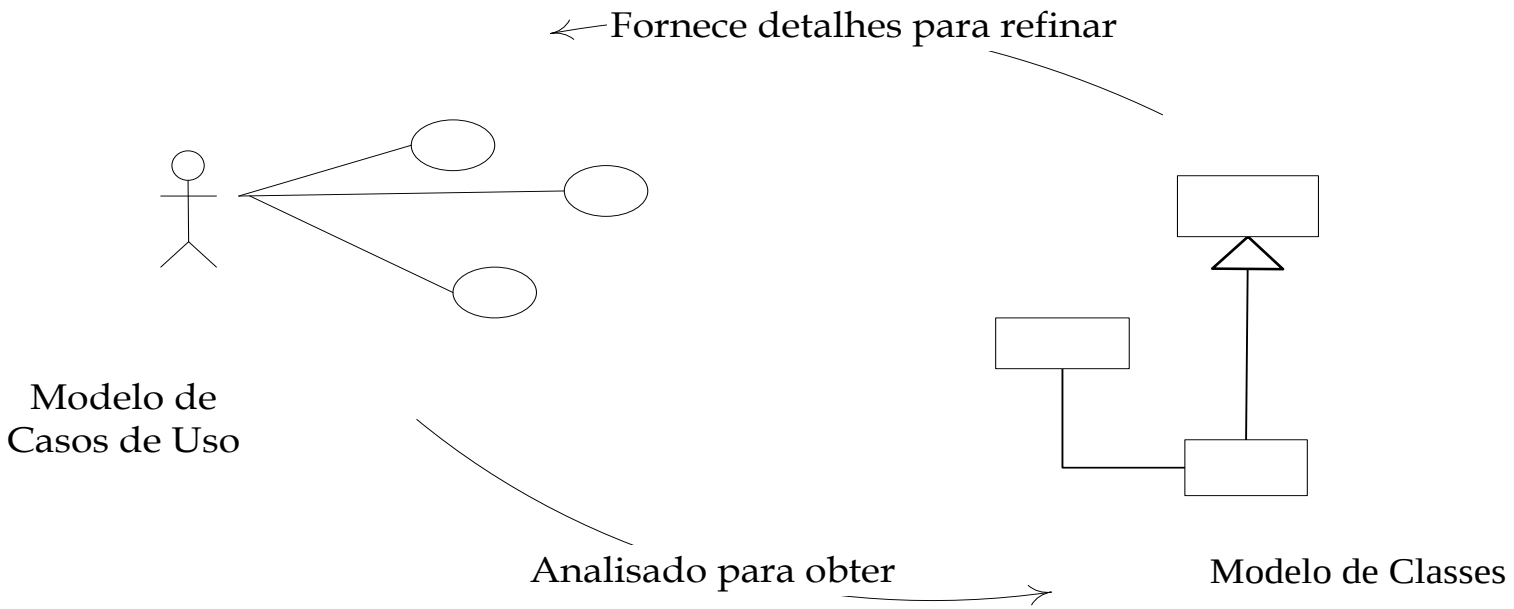


Diagrama de objetos

Diagrama de objetos

- Além do diagrama de classes, A UML define um segundo tipo de diagrama estrutural, o diagrama de objetos.
- Pode ser visto com uma instância de diagramas de classes
- Representa uma “fotografia” do sistema em um certo momento.
  - exibe as ligações formadas entre objetos conforme estes interagem e os valores dos seus atributos.

2006
©MJRL

Engenharia de Software

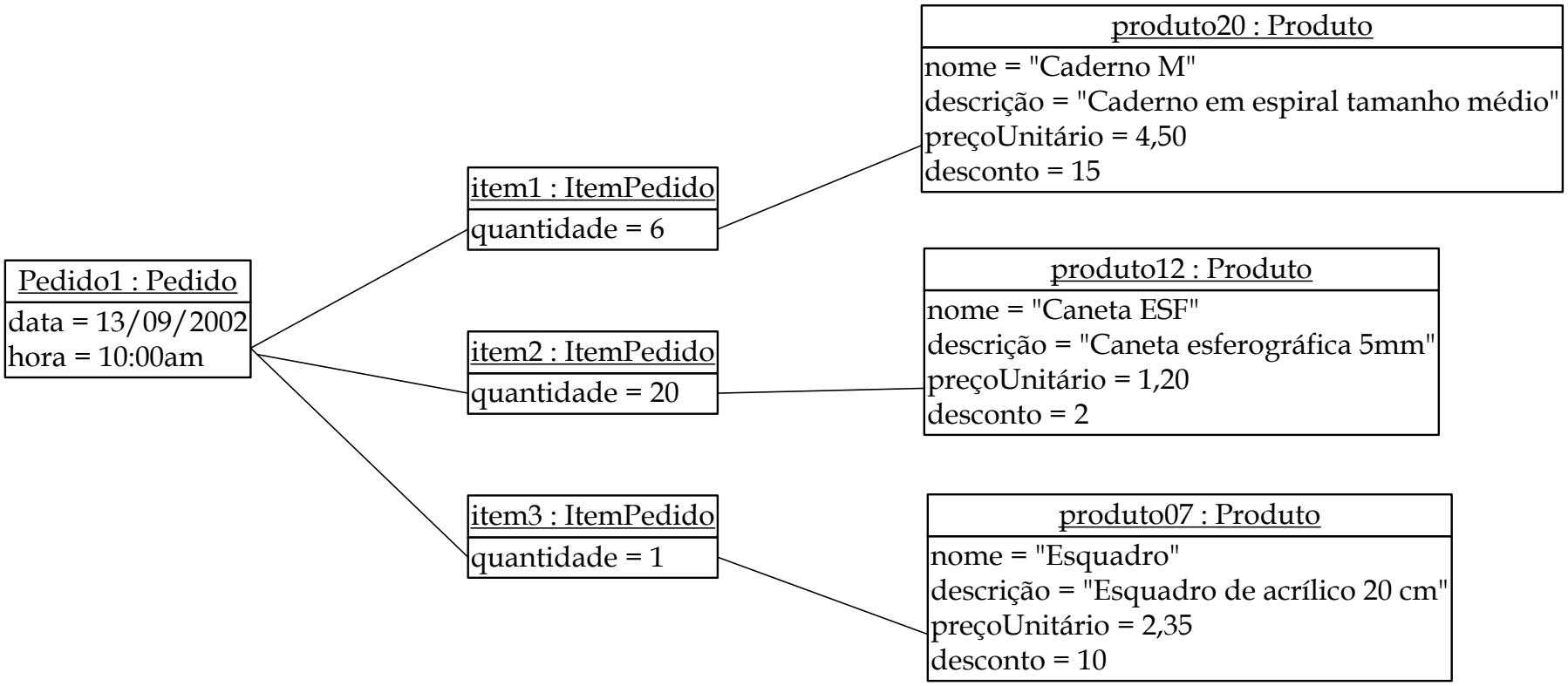
Notação para Diagrama de objetos

Formato	**Exemplo**
nomeClasse	Pedido
nomeObjeto: NomeClasse	umPedido: Pedido

©MJRL

Engenharia de Software

Exemplo (Diagrama de objetos)



2006
©MJRL

Engenharia de Software

Exemplo (Diagrama de objetos)

